Process & Decision Documentation

Project/Assignment Decisions

- At one stage, attempts were made to implement a slash effect that caused the game to freeze ot disappear due to rendering conflicts. Instead of continuing with the complex implementation, I decided to simplify the visual effects and start with basic visuals, such as the trails, and adjusted the boar knockback distance. Instead of increasing knockback strength, the knockback duration(knocktimer) was extended to allow enemies to travel farther
- Another key decision was where sounds should be triggered in the code. Instead of playing sounds from the input system, sounds were placed at meaningful game play events, such as under the space attack key, I would load the swing.mp3 as swing. play

Side Quests and A4 (Individual Work)

- One key decision I made was that I was choosing which files to modify when adding new features like sound effects and gameplay adjustments. I added and loaded sound effects in the assetloader file, which made it organized and easy to access, like “hit enemy”, collectLeaf, and recieveDmage
- Instead of modifying the boat.js to adjust knockback behaviour and rewriting the combat system, I increased the knockback duration so enemies travel farther when hit.
- When loading in spritesheets to replace the previous spritesheet, the increments between each space did not match the old sprite, so there were glitches. For a quick fix, I adjusted the row by adding a decimal like 2.5, so it falls on the ground properly.
- When publishing onto github, the music was not transferring, so I debugged that it was because the p5.js sound was not loaded into the HTML, which couldn’t play the audio.

Role-Based Process Evidence

```
    kb.presses("space")
  ) {
    attackHitThisSwing = false;
    attacking = true;
    attackHitThisSwing = false;
    attackFrameCounter = 0;      You, 2 days ago • fix
    player.vel.x = 0;
    player.ani.frame = 0;
    player.ani = "attack";
    player.ani.play();
    // --- PLAY SWING SOUND --//
    if (swing) swing.play();
  }
```

```
    x = leaf collectible (boars pass through)
    f = fire hazard (player takes damage, boars turn around if the
      = empty (no sprite)
  */
  let level = [
    "                    g   g   b    x       ", // row  0 (2)
    "                b             LggR      ", // row  1 (2)
    "      x     x    LggR    x               ", // row  2
    "     LR   LgR          LR               ", // row  3 (1)
    "   x  b        x   b  x                 ", // row  4
    "   LgggR       LR   LgR        b x       ", // row  5 (3)
    "          LgR   x          ggg   LggggR   ", // row  6 (3)
    "  x           LgR                    x   ", // row  7
    " LgR      xb                    x       LggR", // row  8 (1)
    "         LgR        x       LR  LgR    x ", // row  9
    "   x     [d]       LggR            [dd]", // row 10 (2)
    "LgggRLgggggggR      Lgggg] dd     Lgggggggg", // row 11 (1)
    "dddddddddddddddddddddddddddddddddddddddddd", // row 12
  ];

  // --- LEVEL CONSTANTS ---
```

```
// sprites
this.sprite = null;
this.sensor = null;

// simple visual effect
this.trailTimer = 0; // frames left to draw trail      You,
this.trailPositions = []; // store recent positions

// spawn
const ps = this.levelData.playerStart || {
```

```
// timers      You, 3 days ago • files added …
tickTimers() {
  if (this.invulnTimer > 0) this.invulnTimer--;
  if (this.knockTimer > 0) this.knockTimer--;
  // update trail positions
  // ----------------------
  this.trailTimer++;
  if (this.trailTimer % 2 === 0) {
    // every 2 frames
    // store the current position
    this.trailPositions.push({ x: this.sprite.x, y: this.sprite.y });
    // keep last 6 positions max
    if (this.trailPositions.length > 6) this.trailPositions.shift();
  }
}
```

GenAI Documentation

GenAi was used for this project.

Date Used: March 7, 2026

Tool Disclosure: ChatGPT (GPT-5.3)

Purpose of Use: I used ChatGPT primarily for debugging javascript issues, understanding code errors, and geting guidance on where to place new features in the game files.

Summary of Interaction: The tool helped explain how different parts of the game code functioned and suggested ways to implement features such as sound effects, collision/physics logic for trailing, loading and aligning sprite frame animations.It also provided examples on how to structure sound playback using the asset loader, and where to place sounds to trigger during gameplay events.

Human Decision Point(s): I decided where the suggested code should actually be placed in my project files and adjusted some of the suggestions to match my existing code structure. For instance, I chose to play the candy collection sound inside the leaf collection function instead of in the collision check so the sound would only play when the candy was successfully collected. I also simplified some suggested implementations when earlier attempts caused the game to freeze.

Integrity & Verification Note:

I verified the suggestions by testing the code directly in the game, checking whether the feature worked as expected (for example, confirming that sounds played at the correct events, or if it even made any sounds) if the game broke, I modified or removed the code until it worked correctly.

Scope of GenAI Use:

The existing code and gameplay were already provided and implemented as part of the sidequest. One of the assets, the player, was generated with AI as I wanted a specific character. I mainly used GenAI to debug problems, understand existing systems, and add small features like sound triggers, and debugging the sprites load-in.

Limitations or Misfires: AI could not fully understand the entire game code structure as it was split into multiple working files and sometimes they thought the code was more condensed into one a few js files which made the code much more complex and redundant. Thus, the suggestions causes the game to freeze or screen disppearing. I removed the changes and restrarted from a stable version of the code.

Summary of Process (Human + Tool)

Describe what you did, focusing on process rather than outcome. This may include:

The parts that I implemented mostly myself were loading in the new audio files, increasing the values for knockback and knockback timer, attempting the loop for creating the affect. I tested these small code changes to see if the game still ran, and further asked AI what I was missing and where I went wrong when I did not get the right outcomes. Ai helped debug my trailing effect, and guided where to place codes such as the sound effect for leaf collection and enemy hits.

Decision Points & Trade-offs

Describe one or two key decisions you made:

- I simplified the slash visual effect after it caused the gae to freeze. I ended dup removing the effect due to it not working, and focuses on improving gameplay feedback through sound effects and knockback adjustments
- One sound that was not included was a general “hit” sound effect when pressing space, so I loaded a sound in from pixabay and loaded it in myself. I think it was finethat the swing sound played at the sametime as the “hitenemy” sound as that’s how physics should work.

Verification & Judgement

I I evaluated changes mainly through playtesting the game after each iteration and change. If a feature worked, I kept it. I also checked that sounds triggered at the correct moements and adjusted it if it didn't. I compared results with assignment requirements to see if I've learned and applied what I should have for this sidequest.

Limitations, Dead Ends, or Open Questions

- The last hit for the "hitenemy" sound effect had no sound, I think it triggered as 'defeat" and not an enemy hit.

Appendix

Lisa: I'm trying to add `music.wav` to my game, but it's breaking and won't load in `setup`.

ChatGPT: The issue is probably when you're trying to play it. Modern browsers block autoplay for audio. A good approach is to **preload** the file and play it only after the player interacts.

Example code:

```
let music;

function preload() {
 music = loadSound('assets/music.wav');
}

function setup() {
 createCanvas(800, 600);
}

function keyPressed() {
 if (!music.isPlaying()) {
   music.play();
 }
}
```

Lisa: Can we do it normally, without any "safe mode" stuff?

ChatGPT: Yes, just make sure the `.play()` call happens in a **user-triggered function**, like `keyPressed()` or `mousePressed()`. That way it works reliably in VS Code and in the browser.

Lisa: I want a trailing effect for the player. How should I do it?

ChatGPT: You can store the **recent positions** of the player in an array and draw them faintly to create a trail. Limit the array so older positions fade out.

```
let trail = [];
let maxTrail = 10;

function draw() {
 background(0);

 // Add current position to trail
 trail.push({x: player.x, y: player.y});
 if (trail.length > maxTrail) {
   trail.shift(); // remove oldest
 }

 // Draw trail
 for (let i = 0; i < trail.length; i++) {
   fill(255, i * 25);
   ellipse(trail[i].x, trail[i].y, 20, 20);
 }

 player.update();
 player.show();
}
```

Lisa: My current sprite is a fox, but I downloaded a cowgirl sprite as individual frames. Can I use this?

ChatGPT: You **can**, but it's more complicated than a sprite sheet. You'd have to load each frame individually and manage the timing manually.

```
let frames = [];
let currentFrame = 0;

function preload() {
 for (let i = 1; i <= 6; i++) {
   frames.push(loadImage(`assets/cowgirl/frame${i}.png`));
 }
}

function draw() {
 image(frames[currentFrame], player.x, player.y);
```

```
 if (frameCount % 6 === 0) currentFrame = (currentFrame + 1) % frames.length;
}
```

ChatGPT: But using a **single sprite sheet** is simpler. You just slice the sheet by width/height and cycle through frames, avoiding misalignment.

```
let spriteSheet;
let frameWidth = 64;
let frameHeight = 64;
let currentFrame = 0;
let totalFrames = 6;

function preload() {
 spriteSheet = loadImage('assets/cowgirl_sheet.png');
}

function draw() {
 let sx = (currentFrame % totalFrames) * frameWidth;
 let sy = 0;
 image(spriteSheet, player.x, player.y, frameWidth, frameHeight, sx, sy, frameWidth,
frameHeight);
 if (frameCount % 6 === 0) currentFrame = (currentFrame + 1) % totalFrames;
}
```

Lisa: How do I debug sound not playing when I publish?

ChatGPT: Check three things:

1. Make sure the **path is correct**.
2. `.play()` is called **after user interaction**.
3. Check the **browser console** for errors — often it's autoplay policy or file location.

Lisa: I also want to add different tile/platform layouts.

ChatGPT: A good approach is a **character map**, where each character represents a tile type. Loop through the map and draw the tiles in the world.

```
let level = [
 "GGGGGGGGGG",
 "G        G",
```

```
 "G   P    G",
 "GGGGGGGGGG"
];

function drawLevel() {
 for (let y = 0; y < level.length; y++) {
   for (let x = 0; x < level[y].length; x++) {
     let tile = level[y][x];
     if (tile === "G") fill(139, 69, 19); // ground
     else if (tile === "P") fill(0, 255, 0); // platform
     else fill(0); // empty
     rect(x * 64, y * 64, 64, 64);
   }
 }
}
```

ChatGPT: This way, you can create multiple layouts easily by just changing the character map. Make sure the collision boxes match the tiles for smooth gameplay.